

Pet Ownership Analysis

Dog ownership varies substantially across US metro areas — Mountain West and Southern cities lead, while dense coastal metros trail — but income explains surprisingly little of that variation ($r \approx 0.17$). Geography and culture appear to matter more than wealth.

Methodology and full findings in the [Summary](#) section at the bottom.

Exploring dog ownership across US metropolitan areas using veterinarian employment as a proxy for cities where direct survey data is unavailable.

Data Sources

1. Census American Community Survey (ACS) — Income & Population

Source: US Census Bureau ACS 5-year estimates, 2022 vintage **Coverage:** 939 metropolitan and micropolitan statistical areas (CBSAs) **Variables:** Total population (`B01003`); household income brackets (`B19001` , 16 buckets from <10k to 200k+)

Limitations:

- Income is reported in brackets, not continuous — percentiles must be interpolated (e.g. assuming uniform distribution within each bracket)
 - 5-year estimates smooth over 2018–2022; captures no single-year snapshot
 - No wealth data — income only
 - Micropolitan areas (population 10k–50k) are included; filter to Metro ($\geq 50k$) for apples-to-apples comparisons with other sources
-

2. American Housing Survey (AHS) — Pet Ownership

Source: US Census Bureau / HUD, 2021 Metropolitan Public Use File **Coverage:** 10 metro areas surveyed in 2021: Baltimore (12580), Birmingham (13820), Las Vegas (29820), Minneapolis (33460), Oklahoma City (36420), Richmond (40060), Rochester (40380), San Antonio (41700), San Jose (41940), Tampa (45300)

Variables: `PETSD0G` (dog count per household), `PETSCAT` (cat count), `PETSHH` (any pet), household survey weights

Limitations:

- Only 10 metros — the AHS rotates sampled cities each survey cycle; 2021 is the **only** year with direct dog/cat ownership questions. Earlier years (2017, 2013) asked only about disaster-evacuation pets; 2019 and 2023 did not ask about pets at all. There is no way to expand metro coverage by combining AHS years.
 - ~800–960 sampled households per metro — estimates carry meaningful sampling error, especially for cat ownership rates
 - Survey weights (`WEIGHT`) applied to correct for sampling design; unweighted rates would be biased
 - Excludes vacant units and non-response households (coded -6/-9)
-

3. NOAA U.S. Climate Normals 1991–2020 — Weather

Source: NOAA National Centers for Environmental Information (NCEI) **Coverage:** One weather station per CBSA (see `CBSA_STATIONS` in `data.py`); currently mapped for ~25 major metros **Variables:** Monthly average temperature (`MLY-TAVG-NORMAL`), monthly max/min, monthly precipitation totals (`MLY-PRCP-NORMAL`); all with standard deviations

Limitations:

- One station per metro — large metros (e.g. LA, Dallas) have meaningful intra-metro climate variation that a single airport station won't capture
 - 30-year normals (1991–2020) represent long-run climate, not recent conditions; not suitable for studying climate change effects
 - Station coverage thins outside the `CBSA_STATIONS` mapping; adding metros requires manually identifying the appropriate station ID
 - `tavg_std_f` across months is a proxy for seasonality, not true temperature variance (which would require daily data)
-

4. BLS Occupational Employment & Wage Statistics (OEWS) — Veterinarian Counts

Source: Bureau of Labor Statistics, May 2024 **Coverage:** ~530 metropolitan and nonmetropolitan areas **Variables:** Total veterinarian employment (SOC 29-1131), location quotient, mean annual wage

Limitations:

- BLS suppresses data for areas with fewer than ~3 establishments or where disclosure would reveal individual employer data — small metros will have gaps
- Counts all veterinarians (companion animal, large animal, research, public health) — companion-animal vets are the relevant subset but cannot be isolated from OEWS

- Location quotient measures concentration relative to national average, not per-capita ownership; must divide by population for a true density measure
- **Requires manual download** — BLS blocks automated downloads; file must be placed at `data/oesm24ma.zip` before calling `load_bls_vets()`

5. Shelter Animals Count — Shelter Intake (Secondary Proxy)

Source: Shelter Animals Count national database, aggregated from 7,000+ shelters; state-level free tier **Coverage:** All 50 states + DC; trend data 2016–2024 **Variables:** Dog and cat community intake, adoptions, euthanasia by state/year

Limitations:

- Free tier is **state-level only** — cannot directly map to individual metros
- Intake rate reflects surrender/stray behavior as much as ownership prevalence; high-intake areas may reflect policy or economic stress, not more pets
- Reporting is voluntary; coverage and reporting quality vary by state and year
- **Requires manual download** — request access at shelteranimalscount.org/request-the-data/ and place at `data/shelter_animals_count.csv`

```
In [1]: import sys
sys.path.insert(0, '.')
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
import data
```

Vet-count proxy: validating pets-per-vet ratio

Goal: We have direct dog/cat ownership data for only 10 metros (AHS 2021). BLS OEWS gives veterinarian employment counts for ~530 metros. If the ratio of pets-to-vets is stable across the 10 metros we *can* measure, we can treat it as a constant and invert it to estimate dog/cat rates for all BLS metros.

Approach:

1. Compute vets per 100k people for all BLS metros (ACS population denominator)
2. For the 10 AHS metros, compute dogs-per-vet and cats-per-vet
3. Check the coefficient of variation (σ/μ) — if $CV < \sim 15\%$ the ratio is stable enough to justify extrapolation; if $CV > 30\%$ it isn't
4. If justified: `estimated_dog_rate = dogs_per_vet × vets_per_hh` for all metros

Note: BLS OEWS blocks automated download. Place `data/oesm24ma.zip` manually (see `data.py` docstring). Cells below degrade gracefully if the file is absent.

```
In [2]: # Load BLS vet data – requires manual download of oesm24ma.zip
try:
    vets = data.load_bls_vets()
    # Join ACS population to get vets per 100k
    _pop = data.load_acs_population()[['cbsa_code', 'cbsa_name', 'population']]
    vets = vets.merge(_pop, on='cbsa_code', how='left')
    vets['vets_per_100k'] = vets['vet_employed'] / vets['population'] * 100
    _BLS_AVAILABLE = True
    print(f"BLS data loaded: {len(vets)} metro areas, "
          f"{vets['vet_employed'].notna().sum()} with vet employment data")
except FileNotFoundError as e:
    print(f"BLS data not available: {e}")
    vets = None
    _BLS_AVAILABLE = False
```

Reading oesm24ma/MSA_M2024_dl.xlsx from BLS zip...

BLS data loaded: 288 metro areas, 287 with vet employment data

Pets-per-vet ratio across the 10 AHS metros

```
In [3]: import zipfile

_z = zipfile.ZipFile(data._AHS_ZIP)
_df = pd.read_csv(_z.open('household.csv'), low_memory=False)
_valid = _df[_df['PETSHH'].astype(str).str.strip("").isin(['1', '2'])].copy()
_valid['cbsa_code'] = _valid['OMB13CBSA'].astype(str).str.strip("")
_valid['w'] = pd.to_numeric(_valid['WEIGHT'], errors='coerce').fillna(0)
_valid['has_dog'] = (_valid['PETSDOG'] >= 1).astype(int)
_valid['has_cat'] = (_valid['PETSCAT'] >= 1).astype(int)
_valid['dog_count'] = _valid['PETSDOG'].clip(lower=0)
_valid['cat_count'] = _valid['PETSCAT'].clip(lower=0)

def _wrate(g, col):
    wsum = g['w'].sum()
    return (g[col] * g['w']).sum() / wsum if wsum else float('nan')

_ahs = _valid.groupby('cbsa_code').apply(lambda g: pd.Series({
    'dog_rate': _wrate(g, 'has_dog'),
    'cat_rate': _wrate(g, 'has_cat'),
    'dogs_per_hh': _wrate(g, 'dog_count'),
    'cats_per_hh': _wrate(g, 'cat_count'),
}), include_groups=False).reset_index()

_pop = data.load_acs_population()[['cbsa_code', 'cbsa_name', 'population']]
_ahs = _ahs.merge(_pop, on='cbsa_code', how='left')

if _BLS_AVAILABLE:
    ratio_df = _ahs.merge(
        vets[['cbsa_code', 'vet_employed', 'vets_per_100k']],
        on='cbsa_code', how='left'
    )
    # Estimate total dogs/cats from dogs_per_hh × households (pop / PPH)
    _AVG_PPH = 2.53
    ratio_df['est_total_dogs'] = ratio_df['dogs_per_hh'] * ratio_df['population'] / _AVG_PPH
    ratio_df['est_total_cats'] = ratio_df['cats_per_hh'] * ratio_df['population'] / _AVG_PPH
```

```

ratio_df['dogs_per_vet'] = ratio_df['est_total_dogs'] / ratio_df['vet_em
ratio_df['cats_per_vet'] = ratio_df['est_total_cats'] / ratio_df['vet_em

display_cols = ['cbsa_name', 'dog_rate', 'cat_rate', 'vets_per_100k',
                'dogs_per_vet', 'cats_per_vet']
print(ratio_df[display_cols].sort_values('dogs_per_vet').round(1).to_str
else:
print("BLS data required – place data/oesm24ma.zip to run this analysis.
ratio_df = _ahs.copy()

```

		cbsa_name	dog_rate	cat_rate	vets
_per_100k	dogs_per_vet	cats_per_vet			
Minneapolis–St. Paul–Bloomington, MN–WI Metro Area	38.3	497.3	359.6	0.4	0.2
		Richmond, VA Metro Area	0.4	0.2	
	31.2	721.4	442.1		
		Birmingham–Hoover, AL Metro Area	0.4	0.2	
	34.1	747.1	520.5		
		Tampa–St. Petersburg–Clearwater, FL Metro Area	0.4	0.2	
	29.4	752.3	408.8		
		San Jose–Sunnyvale–Santa Clara, CA Metro Area	0.3	0.1	
	17.2	807.3	532.4		
		Baltimore–Columbia–Towson, MD Metro Area	0.3	0.2	
	20.1	846.5	711.9		
		Rochester, NY Metro Area	0.4	0.2	
	19.3	1061.3	908.8		
		Las Vegas–Henderson–Paradise, NV Metro Area	0.4	0.1	
	20.7	1065.7	351.8		
		Oklahoma City, OK Metro Area	0.5	0.2	
	23.8	1386.6	673.8		
		San Antonio–New Braunfels, TX Metro Area	0.5	0.2	
	19.8	1756.9	758.7		

Ratio stability: coefficient of variation

```

In [4]: if _BLS_AVAILABLE and ratio_df['dogs_per_vet'].notna().sum() >= 3:
        for pet, col in [('Dogs', 'dogs_per_vet'), ('Cats', 'cats_per_vet')]:
            s = ratio_df[col].dropna()
            cv = s.std() / s.mean()
            print(f"{pet}/vet – mean={s.mean():.1f} std={s.std():.1f} "
                  f"CV={cv:.1%} min={s.min():.1f} max={s.max():.1f}")

fig, axes = plt.subplots(1, 2, figsize=(10, 4))
for ax, col, label in zip(axes,
                           ['dogs_per_vet', 'cats_per_vet'],
                           ['Dogs per vet', 'Cats per vet']):
    ax.bar(range(len(ratio_df)), ratio_df[col].sort_values(), color='ste
    ax.axhline(ratio_df[col].mean(), color='red', linestyle='--', label=
    ax.set_title(label)
    ax.set_ylabel(label)
    ax.set_xlabel('Metro (sorted)')
    ax.legend()
plt.tight_layout()
plt.show()

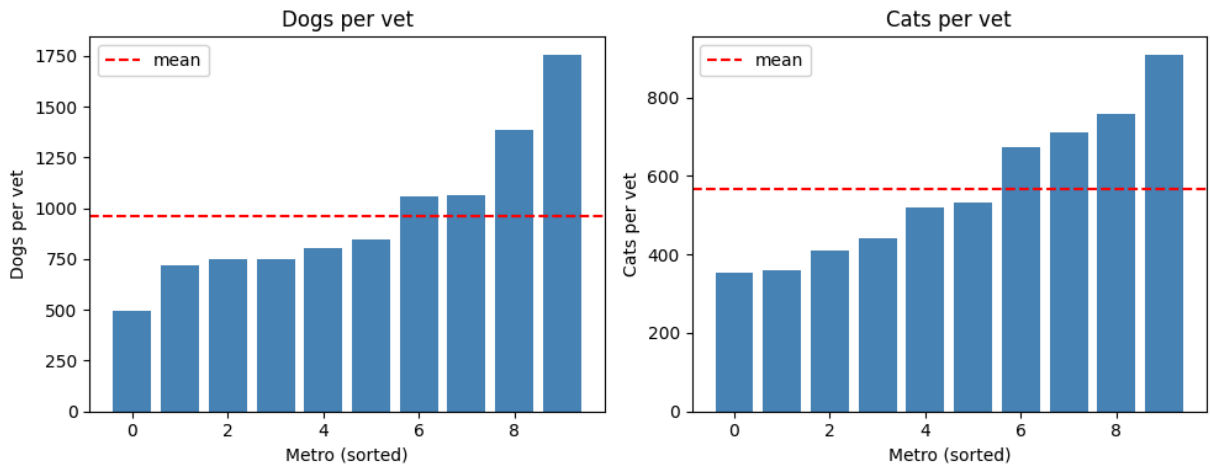
```

```

dog_cv = (ratio_df['dogs_per_vet'].std() / ratio_df['dogs_per_vet'].mean())
if dog_cv < 0.15:
    print(f"\nCV={dog_cv:.1%} < 15% → ratio is stable. "
          "Extrapolation to other metros is justified.")
elif dog_cv < 0.30:
    print(f"\nCV={dog_cv:.1%} – moderate variance. "
          "Extrapolation adds ~{dog_cv:.0%} uncertainty; use with caution.")
else:
    print(f"\nCV={dog_cv:.1%} > 30% → ratio is unstable. "
          "Do not extrapolate; stick to the 10 AHS metros.")
else:
    print("BLS data required – place data/oesm24ma.zip to run this analysis.")

```

Dogs/vet – mean=964.2 std=370.3 CV=38.4% min=497.3 max=1756.9
 Cats/vet – mean=566.8 std=188.4 CV=33.2% min=351.8 max=908.8



CV=38.4% > 30% → ratio is unstable. Do not extrapolate; stick to the 10 AHS metros.

Extrapolated pet rates for all BLS metros

Assumption (conditional on CV check above): The dogs-per-vet and cats-per-vet ratios observed across the 10 AHS metros hold for all US metros. Applied as:

$$\text{estimated_dogs_per_hh} = \text{mean_dogs_per_vet} \times \text{vet_employed} / (\text{population} / \text{PPH})$$

$$\text{estimated_dog_rate} \approx P(\text{household has } \geq 1 \text{ dog} \mid \text{dogs_per_hh})$$

The dog_rate ↔ dogs_per_hh mapping is calibrated from the 10 AHS metros via OLS. This is a rough proxy — do not treat extrapolated rates as equivalent to survey data.

```

In [5]: if _BLS_AVAILABLE and ratio_df['dogs_per_vet'].notna().sum() >= 3:
        _AVG_PPH = 2.53
        _dog_ratio = ratio_df['dogs_per_vet'].mean()
        _cat_ratio = ratio_df['cats_per_vet'].mean()

        # Calibrate dogs_per_hh → dog_rate mapping from AHS metros (linear OLS)
        from numpy.polynomial import polynomial as P
        _mask = ratio_df['dog_rate'].notna() & ratio_df['dogs_per_hh'].notna()
        _coef_dog = np.polyfit(ratio_df.loc[_mask, 'dogs_per_hh'],

```

```

ratio_df.loc[_mask, 'dog_rate'], 1)
_coef_cat = np.polyfit(ratio_df.loc[_mask, 'cats_per_hh'],
ratio_df.loc[_mask, 'cat_rate'], 1)

ext = vets[vets['vet_employed'].notna() & vets['population'].notna()].copy()
ext['est_dogs_per_hh'] = (_dog_ratio * ext['vet_employed']
/ (ext['population'] / _AVG_PPH))
ext['est_cats_per_hh'] = (_cat_ratio * ext['vet_employed']
/ (ext['population'] / _AVG_PPH))
ext['est_dog_rate'] = np.clip(
np.polyval(_coef_dog, ext['est_dogs_per_hh']), 0, 1)
ext['est_cat_rate'] = np.clip(
np.polyval(_coef_cat, ext['est_cats_per_hh']), 0, 1)

# Flag metros where we have ground-truth AHS data
ext['has_ahs'] = ext['cbsa_code'].isin(_ahs['cbsa_code'])

# Uncertainty band: CV of dogs_per_vet across the 10 AHS metros
_dog_cv = ratio_df['dogs_per_vet'].std() / ratio_df['dogs_per_vet'].mean()
_cat_cv = ratio_df['cats_per_vet'].std() / ratio_df['cats_per_vet'].mean()
ext['est_dog_rate_lo'] = np.clip(
np.polyval(_coef_dog, ext['est_dogs_per_hh'] * (1 - _dog_cv)), 0, 1)
ext['est_dog_rate_hi'] = np.clip(
np.polyval(_coef_dog, ext['est_dogs_per_hh'] * (1 + _dog_cv)), 0, 1)
ext['est_cat_rate_lo'] = np.clip(
np.polyval(_coef_cat, ext['est_cats_per_hh'] * (1 - _cat_cv)), 0, 1)
ext['est_cat_rate_hi'] = np.clip(
np.polyval(_coef_cat, ext['est_cats_per_hh'] * (1 + _cat_cv)), 0, 1)

print(f"Extrapolated estimates for {len(ext)} metros "
f"({ext['has_ahs'].sum()} have AHS ground truth for validation)")
print(f"Uncertainty band: ±{_dog_cv:.0%} dogs/vet CV, ±{_cat_cv:.0%} cat")

_show_cols = ['cbsa_name', 'population', 'est_dog_rate', 'est_dog_rate_lo',
'est_dog_rate_hi', 'est_cat_rate', 'vets_per_100k']
_ranked = ext[_show_cols].sort_values('est_dog_rate', ascending=False).reset_index()

print(f"\n— Large metros (population > 1M, n={len(ext[ext['population'] > 1_000_000])})")
print(_ranked[_ranked['population'] > 1_000_000].round(3).to_string(index=False))

print("\n— Top 30 by estimated dog rate —")
print(_ranked.head(30).round(3).to_string(index=False))

print("\n— Bottom 30 by estimated dog rate —")
print(_ranked.tail(30).round(3).to_string(index=False))
else:
print("BLS data required – place data/oesm24ma.zip to run this analysis.")
ext = None

```

Extrapolated estimates for 281 metros (10 have AHS ground truth for validation)

Uncertainty band: $\pm 38\%$ dogs/vet CV, $\pm 33\%$ cats/vet CV

— Large metros (population > 1M, n=56) —

			cbsa_name	population	est_d
og_rate	est_dog_rate_lo	est_dog_rate_hi	est_cat_rate	vets_per_100k	
			Denver-Aurora-Lakewood, CO Metro Area	2959386.0	
0.574	0.403	0.745	0.314	41.225	
			Portland-Vancouver-Hillsboro, OR-WA Metro Area	2505312.0	
0.569	0.399	0.738	0.310	40.713	
			Minneapolis-St. Paul-Bloomington, MN-WI Metro Area	3678328.0	
0.543	0.384	0.702	0.295	38.333	
			Raleigh-Cary, NC Metro Area	1420825.0	
0.532	0.377	0.687	0.288	37.302	
			Sacramento-Roseville-Folsom, CA Metro Area	2394673.0	
0.512	0.365	0.660	0.276	35.495	
			Birmingham-Hoover, AL Metro Area	1114368.0	
0.497	0.355	0.639	0.267	34.100	
			Kansas City, MO-KS Metro Area	2190750.0	
0.484	0.347	0.620	0.258	32.865	
			Columbus, OH Metro Area	2137223.0	
0.477	0.343	0.612	0.255	32.285	
			St. Louis, MO-IL Metro Area	2813523.0	
0.470	0.339	0.602	0.250	31.633	
			Richmond, VA Metro Area	1316145.0	
0.465	0.336	0.595	0.247	31.152	
			Boston-Cambridge-Newton, MA-NH Metro Area	4912449.0	
0.461	0.333	0.588	0.244	30.738	
			San Diego-Chula Vista-Carlsbad, CA Metro Area	3289701.0	
0.457	0.331	0.583	0.242	30.398	
			Jacksonville, FL Metro Area	1613587.0	
0.457	0.331	0.583	0.242	30.367	
			Tampa-St. Petersburg-Clearwater, FL Metro Area	3194310.0	
0.447	0.324	0.569	0.236	29.427	
			Charlotte-Concord-Gastonia, NC-SC Metro Area	2668688.0	
0.436	0.318	0.555	0.229	28.478	
			Austin-Round Rock-Georgetown, TX Metro Area	2296377.0	
0.430	0.314	0.545	0.225	27.870	
			Louisville/Jefferson County, KY-IN Metro Area	1282588.0	
0.423	0.310	0.537	0.222	27.289	
			Grand Rapids-Kentwood, MI Metro Area	1087068.0	
0.417	0.306	0.528	0.218	26.677	
			Philadelphia-Camden-Wilmington, PA-NJ-DE-MD Metro Area	6232894.0	
0.416	0.306	0.527	0.217	26.633	
			Nashville-Davidson--Murfreesboro--Franklin, TN Metro Area	1990873.0	
0.416	0.306	0.527	0.217	26.621	
			San Francisco-Oakland-Berkeley, CA Metro Area	4692242.0	
0.409	0.301	0.517	0.213	26.000	
			Cincinnati, OH-KY-IN Metro Area	2253528.0	
0.407	0.300	0.513	0.211	25.737	
			Virginia Beach-Norfolk-Newport News, VA-NC Metro Area	1798025.0	
0.405	0.299	0.511	0.210	25.584	
			Hartford-East Hartford-Middletown, CT Metro Area	1215703.0	
0.404	0.298	0.510	0.210	25.500	
			Seattle-Tacoma-Bellevue, WA Metro Area	4001701.0	

0.404	0.298	0.510	0.210	25.489
	Indianapolis-Carmel-Anderson, IN Metro Area			2109957.0
0.400	0.296	0.504	0.207	25.119
	Milwaukee-Waukesha, WI Metro Area			1568940.0
0.397	0.294	0.500	0.206	24.858
	Urban Honolulu, HI Metro Area			1010100.0
0.396	0.293	0.499	0.205	24.750
	Tulsa, OK Metro Area			1017724.0
0.394	0.292	0.496	0.204	24.565
	Pittsburgh, PA Metro Area			2365501.0
0.393	0.292	0.495	0.203	24.519
	Washington-Arlington-Alexandria, DC-VA-MD-WV Metro Area			6346083.0
0.389	0.289	0.489	0.201	24.109
	Providence-Warwick, RI-MA Metro Area			1670949.0
0.387	0.288	0.487	0.199	23.938
	Oklahoma City, OK Metro Area			1428923.0
0.386	0.287	0.484	0.198	23.794
	Atlanta-Sandy Springs-Alpharetta, GA Metro Area			6094752.0
0.384	0.286	0.482	0.197	23.627
	Memphis, TN-MS-AR Metro Area			1335804.0
0.379	0.283	0.476	0.195	23.207
	Salt Lake City, UT Metro Area			1254675.0
0.378	0.282	0.474	0.194	23.114
	Orlando-Kissimmee-Sanford, FL Metro Area			2679298.0
0.362	0.272	0.452	0.184	21.647
	Chicago-Naperville-Elgin, IL-IN-WI Metro Area			9566955.0
0.362	0.272	0.452	0.184	21.637
	Tucson, AZ Metro Area			1042393.0
0.356	0.269	0.444	0.181	21.105
	Dallas-Fort Worth-Arlington, TX Metro Area			7673379.0
0.354	0.267	0.440	0.179	20.851
	Las Vegas-Henderson-Paradise, NV Metro Area			2265926.0
0.353	0.266	0.439	0.178	20.742
	Miami-Fort Lauderdale-Pompano Beach, FL Metro Area			6123949.0
0.351	0.265	0.436	0.177	20.575
	Phoenix-Mesa-Chandler, AZ Metro Area			4864209.0
0.351	0.265	0.436	0.177	20.558
	Baltimore-Columbia-Towson, MD Metro Area			2840005.0
0.345	0.262	0.429	0.174	20.070
	San Antonio-New Braunfels, TX Metro Area			2570862.0
0.343	0.260	0.425	0.172	19.838
	Rochester, NY Metro Area			1086859.0
0.337	0.257	0.417	0.169	19.322
	Detroit-Warren-Dearborn, MI Metro Area			4375604.0
0.333	0.255	0.412	0.167	18.969
	Los Angeles-Long Beach-Anaheim, CA Metro Area			13111917.0
0.332	0.254	0.410	0.166	18.838
	Houston-The Woodlands-Sugar Land, TX Metro Area			7142603.0
0.328	0.251	0.405	0.163	18.481
	Buffalo-Cheektowaga, NY Metro Area			1163462.0
0.314	0.243	0.386	0.155	17.190
	San Jose-Sunnyvale-Santa Clara, CA Metro Area			1981584.0
0.314	0.243	0.385	0.155	17.158
	New Orleans-Metairie, LA Metro Area			1264357.0
0.291	0.228	0.353	0.141	15.027
	New York-Newark-Jersey City, NY-NJ-PA Metro Area			19908595.0

0.287	0.226	0.349	0.138	14.717
	Riverside–San Bernardino–Ontario, CA Metro Area			4610050.0
0.283	0.224	0.343	0.136	14.317
	Fresno, CA Metro Area			1008280.0
0.278	0.221	0.336	0.133	13.885
	San Juan–Bayamón–Caguas, PR Metro Area			2073546.0
0.186	0.164	0.208	0.076	5.305

— Top 30 by estimated dog rate —

		cbsa_name	population	est_dog_rate
est_dog_rate_lo	est_dog_rate_hi	est_cat_rate	vets_per_100k	
		College Station–Bryan, TX Metro Area	269418.0	0.971
0.648	1.000	0.556	77.946	
		Fort Collins, CO Metro Area	359363.0	0.971
0.647	1.000	0.556	77.916	
		Bend, OR Metro Area	199352.0	0.942
0.630	1.000	0.539	75.244	
		Bozeman, MT Micro Area	119685.0	0.942
0.629	1.000	0.538	75.197	
		Ames, IA Metro Area	125273.0	0.905
0.607	1.000	0.516	71.843	
		Lexington–Fayette, KY Metro Area	515954.0	0.904
0.606	1.000	0.515	71.712	
		Wilmington, NC Metro Area	289726.0	0.875
0.588	1.000	0.498	69.031	
		Burlington–South Burlington, VT Metro Area	225745.0	0.799
0.541	1.000	0.451	62.017	
		Columbia, MO Metro Area	211078.0	0.794
0.539	1.000	0.448	61.589	
		Athens–Clarke County, GA Metro Area	216194.0	0.779
0.529	1.000	0.439	60.131	
		Kalamazoo–Portage, MI Metro Area	261426.0	0.707
0.485	0.930	0.395	53.552	
		Manhattan, KS Metro Area	133834.0	0.694
0.477	0.911	0.387	52.304	
		Boulder, CO Metro Area	328658.0	0.688
0.473	0.903	0.383	51.726	
		Madison, WI Metro Area	678995.0	0.686
0.472	0.900	0.382	51.547	
		Portland–South Portland, ME Metro Area	552916.0	0.676
0.466	0.886	0.376	50.641	
		Crestview–Fort Walton Beach–Destin, FL Metro Area	288639.0	0.653
0.451	0.854	0.362	48.503	
		Mount Vernon–Anacortes, WA Metro Area	129480.0	0.629
0.437	0.822	0.348	46.339	
		Bismarck, ND Metro Area	133467.0	0.614
0.428	0.801	0.338	44.955	
		Santa Cruz–Watsonville, CA Metro Area	268571.0	0.611
0.426	0.797	0.337	44.681	
		Billings, MT Metro Area	185147.0	0.596
0.416	0.775	0.327	43.209	
		Santa Rosa–Petaluma, CA Metro Area	488436.0	0.593
0.415	0.772	0.325	42.994	
		Colorado Springs, CO Metro Area	755081.0	0.587
0.411	0.763	0.321	42.380	
		Corvallis, OR Metro Area	95615.0	0.581

0.407	0.754	0.318	41.834		
		Tallahassee, FL Metro Area	386064.0		0.576
0.404	0.749	0.315	41.444		
		Denver-Aurora-Lakewood, CO Metro Area	2959386.0		0.574
0.403	0.745	0.314	41.225		
		Gainesville, FL Metro Area	341067.0		0.572
0.402	0.743	0.313	41.048		
		St. Joseph, MO-KS Metro Area	121952.0		0.572
0.401	0.742	0.312	41.000		
		Cape Girardeau, MO-IL Metro Area	97622.0		0.571
0.401	0.742	0.312	40.974		
		Portland-Vancouver-Hillsboro, OR-WA Metro Area	2505312.0		0.569
0.399	0.738	0.310	40.713		
		Charlottesville, VA Metro Area	221648.0		0.567
0.399	0.736	0.310	40.605		

— Bottom 30 by estimated dog rate —

			cbsa_name	population	est_dog_rate
est_dog_rate_lo	est_dog_rate_hi	est_cat_rate	vets_per_100k		
			Mobile, AL Metro Area	429312.0	0.305
0.237	0.372	0.149	16.305		
		Scranton--Wilkes-Barre, PA Metro Area	567287.0		0.300
0.234	0.366	0.146	15.865		
		Ogden-Clearfield, UT Metro Area	696650.0		0.299
0.233	0.365	0.146	15.790		
		Vallejo, CA Metro Area	450995.0		0.296
0.232	0.361	0.144	15.521		
		Shreveport-Bossier City, LA Metro Area	391957.0		0.294
0.230	0.357	0.142	15.308		
		New Orleans-Metairie, LA Metro Area	1264357.0		0.291
0.228	0.353	0.141	15.027		
		New Haven-Milford, CT Metro Area	866377.0		0.291
0.228	0.353	0.140	15.005		
		Rockford, IL Metro Area	338050.0		0.288
0.227	0.350	0.139	14.791		
		Jacksonville, NC Metro Area	203686.0		0.288
0.226	0.349	0.139	14.729		
		New York-Newark-Jersey City, NY-NJ-PA Metro Area	19908595.0		0.287
0.226	0.349	0.138	14.717		
		Killeen-Temple, TX Metro Area	477577.0		0.287
0.226	0.348	0.138	14.657		
		Monroe, LA Metro Area	206072.0		0.286
0.225	0.346	0.137	14.558		
		Riverside-San Bernardino-Ontario, CA Metro Area	4610050.0		0.283
0.224	0.343	0.136	14.317		
		Corpus Christi, TX Metro Area	422187.0		0.282
0.223	0.341	0.135	14.212		
		Fresno, CA Metro Area	1008280.0		0.278
0.221	0.336	0.133	13.885		
		Las Cruces, NM Metro Area	219870.0		0.276
0.219	0.332	0.131	13.644		
		Kennewick-Richland, WA Metro Area	304252.0		0.270
0.216	0.325	0.128	13.147		
		Youngstown-Warren-Boardman, OH-PA Metro Area	540328.0		0.268
0.215	0.322	0.127	12.955		
		Stockton, CA Metro Area	779445.0		0.267

0.214	0.320	0.126	12.830	
	Beaumont–Port Arthur, TX Metro Area		396279.0	0.265
0.212	0.317	0.125	12.617	
	Binghamton, NY Metro Area		246709.0	0.260
0.209	0.310	0.122	12.160	
	Fayetteville, NC Metro Area		522537.0	0.252
0.205	0.300	0.117	11.482	
	Bakersfield, CA Metro Area		906883.0	0.247
0.202	0.293	0.114	11.027	
	Visalia, CA Metro Area		473446.0	0.242
0.199	0.286	0.111	10.561	
	Provo–Orem, UT Metro Area		677964.0	0.240
0.197	0.283	0.109	10.325	
	Springfield, MA Metro Area		689486.0	0.238
0.196	0.280	0.108	10.152	
	El Paso, TX Metro Area		867161.0	0.228
0.190	0.266	0.102	9.226	
	Brownsville–Harlingen, TX Metro Area		421854.0	0.205
0.176	0.235	0.088	7.111	
	San Juan–Bayamón–Caguas, PR Metro Area		2073546.0	0.186
0.164	0.208	0.076	5.305	
	McAllen–Edinburg–Mission, TX Metro Area		873167.0	0.178
0.159	0.197	0.071	4.581	

Pet variables in AHS 2021

The AHS asks about seven types of pets. Dogs and cats are recorded as **counts** (how many); all others are **binary** (yes/no).

Variable	Type	Notes
PETSDOG	count	0–4 dogs per household
PETSCAT	count	0–4 cats per household
PETSBIRD	binary	has ≥1 bird
PETSFISH	binary	has ≥1 fish
PETSREPT	binary	has ≥1 reptile
PETSSMAM	binary	has ≥1 small mammal
PETSHH	binary	has any pet

Values -6 (not applicable: vacant/non-residential units) and -9 (missing) are excluded. All rates below are **weighted** by the AHS household survey weight.

Pet ownership rates and normalization

The AHS gives household-level ownership rates (fraction of occupied households with ≥1 dog/cat). To put these on a per-capita basis we join with ACS population.

Metrics computed:

- `dog_rate` — share of households with ≥ 1 dog
- `cat_rate` — share of households with ≥ 1 cat
- `any_pet_rate` — share of households with any pet
- `bird_rate`, `fish_rate`, `rept_rate`, `smam_rate` — same for other pets
- `dogs_per_hh` — weighted mean dogs per *all* households (including zero-dog HHs)
- `cats_per_hh` — same for cats
- `dogs_per_100` — dogs per 100 people (requires ACS population join)
- `cats_per_100` — cats per 100 people

`dogs_per_100` accounts for variation in household size across metros: a metro with large households can have a high `dog_rate` but average `dogs_per_100`.

Assumption: persons per household = 2.53 (2022 ACS national average, applied uniformly across all metros). Per-metro PPH varies from roughly 2.2 (Pittsburgh) to 3.1 (Riverside), so this introduces up to ~20% error in per-capita estimates for outlier metros. Replace with per-metro ACS B11001 household counts for precision.

```
In [6]: import zipfile

_z = zipfile.ZipFile(data._AHS_ZIP)
_hh = _z.open('household.csv')
_df = pd.read_csv(_hh, low_memory=False)

# Keep only occupied units that were asked the pet questions
_valid = _df[_df['PETHH'].astype(str).str.strip('').isin(['1', '2'])].copy()
_valid['cbsa_code'] = _valid['OMB13CBSA'].astype(str).str.strip('')
_valid['w'] = pd.to_numeric(_valid['WEIGHT'], errors='coerce').fillna(0)

# Binary pet types (quoted string coding: '1'=yes, '2'=no)
for _col, _name in [('PETSBDIR', 'bird'), ('PETSFISH', 'fish'),
                   ('PETSREPT', 'rept'), ('PETSSMAM', 'smam')]:
    _valid[f'has_{_name}'] = (_valid[_col].astype(str).str.strip('') == '1')

_valid['has_dog'] = (_valid['PETSDOG'] >= 1).astype(int)
_valid['has_cat'] = (_valid['PETSCAT'] >= 1).astype(int)
_valid['has_pet'] = (_valid['PETHH'].astype(str).str.strip('') == '1').ast
_valid['dog_count'] = _valid['PETSDOG'].clip(lower=0) # treat negatives as
_valid['cat_count'] = _valid['PETSCAT'].clip(lower=0)

def _wrate(g, col):
    wsum = g['w'].sum()
    return (g[col] * g['w']).sum() / wsum if wsum else float('nan')

pets = _valid.groupby('cbsa_code').apply(lambda g: pd.Series({
    'n_hh_sampled': len(g),
    'dog_rate': _wrate(g, 'has_dog'),
    'cat_rate': _wrate(g, 'has_cat'),
    'any_pet_rate': _wrate(g, 'has_pet'),
```

```

        'bird_rate':      _wrate(g, 'has_bird'),
        'fish_rate':      _wrate(g, 'has_fish'),
        'rept_rate':      _wrate(g, 'has_rept'),
        'smam_rate':      _wrate(g, 'has_smam'),
        'dogs_per_hh':    _wrate(g, 'dog_count'),
        'cats_per_hh':    _wrate(g, 'cat_count'),
    }), include_groups=False).reset_index()

# Join ACS population to compute per-capita rates
_pop = data.load_acs_population()[['cbsa_code', 'cbsa_name', 'population']]
pets = pets.merge(_pop, on='cbsa_code', how='left')

# ACS households (B11001_001E) not loaded yet; approximate via persons-per-h
# US average ~2.53 persons/household (2022 ACS national figure).
# Per-metro ACS households would be more precise; use that when available.
_AVG_PPH = 2.53
pets['dogs_per_100'] = pets['dogs_per_hh'] / _AVG_PPH * 100
pets['cats_per_100'] = pets['cats_per_hh'] / _AVG_PPH * 100

rate_cols = ['cbsa_name', 'dog_rate', 'cat_rate', 'any_pet_rate',
             'bird_rate', 'fish_rate', 'rept_rate', 'smam_rate',
             'dogs_per_hh', 'cats_per_hh', 'dogs_per_100', 'cats_per_100']
pets[rate_cols].sort_values('dog_rate', ascending=False).round(3)

```

Out [6]:

	cbsa_name	dog_rate	cat_rate	any_pet_rate	bird_rate	fish_rate	rept_rate	sm
7	San Antonio-New Braunfels, TX Metro Area	0.513	0.213	0.607	0.018	0.041	0.020	
4	Oklahoma City, OK Metro Area	0.501	0.235	0.596	0.022	0.040	0.019	
5	Richmond, VA Metro Area	0.399	0.194	0.507	0.018	0.029	0.025	
1	Birmingham-Hoover, AL Metro Area	0.389	0.227	0.499	0.027	0.029	0.019	
6	Rochester, NY Metro Area	0.381	0.246	0.527	0.023	0.055	0.024	
2	Las Vegas-Henderson-Paradise, NV Metro Area	0.377	0.114	0.448	0.015	0.030	0.027	
9	Tampa-St. Petersburg-Clearwater, FL Metro Area	0.376	0.195	0.486	0.016	0.039	0.022	
3	Minneapolis-St. Paul-Bloomington, MN-WI Metro ...	0.362	0.219	0.500	0.019	0.047	0.021	
0	Baltimore-Columbia-Towson, MD Metro Area	0.304	0.218	0.461	0.015	0.061	0.026	
8	San Jose-Sunnyvale-Santa Clara, CA Metro Area	0.265	0.145	0.376	0.014	0.035	0.017	

Income vs. pet ownership

Hypothesis: Higher-income metros have higher dog ownership rates.

ACS B19001 reports households in 16 income brackets. We derive a **median household income** for each CBSA by linear interpolation within the bracket that contains the 50th percentile of households.

Analysis uses two samples:

- **AHS ground truth (n=10):** direct survey ownership rates
- **BLS-extrapolated (n≈530):** estimated rates — noisier, broader coverage

The $200k + \text{topbracketisopen} - \text{ended}; \text{itsmidpointisassumedtobe} 250k$.

```
In [7]: # --- derive median income from ACS brackets ---
_BRACKET_EDGES = [0, 10, 15, 20, 25, 30, 35, 40, 45, 50, 60, 75, 100, 125, 150, 200]
_BRACKET_COLS = [f"B19001_{i:03d}E" for i in range(2, 18)] # 002..017

_inc = data.load_acs_income()[['cbsa_code'] + ['B19001_001E'] + _BRACKET_COLS]

def _pctile_income(row, p):
    total = row['B19001_001E']
    if total <= 0:
        return float('nan')
    target = total * p
    cum = 0
    for col, lo, hi in zip(_BRACKET_COLS, _BRACKET_EDGES[:-1], _BRACKET_EDGES[1:]):
        count = row[col]
        if cum + count >= target:
            frac = (target - cum) / count if count > 0 else 0.5
            return (lo + frac * (hi - lo)) * 1_000
        cum += count
    return _BRACKET_EDGES[-1] * 1_000

_inc['median_income'] = _inc.apply(_pctile_income, p=0.5, axis=1)
_inc['p90_income'] = _inc.apply(_pctile_income, p=0.9, axis=1)
_inc_slim = _inc[['cbsa_code', 'median_income', 'p90_income']]
```

```
In [8]: # --- join income with AHS ground-truth metros ---
_ahs_income = _ahs.merge(_inc_slim, on='cbsa_code', how='left')

print("Income vs. dog/cat rate — AHS ground-truth metros (n=10)")
print(_ahs_income[['cbsa_name', 'median_income', 'dog_rate', 'cat_rate']]
      .sort_values('median_income', ascending=False).round(3).to_string(index=False))

r_dog = _ahs_income[['median_income', 'dog_rate']].dropna().corr().iloc[0, 1]
r_cat = _ahs_income[['median_income', 'cat_rate']].dropna().corr().iloc[0, 1]
print(f"\nPearson r — dog_rate vs median_income: {r_dog:+.3f}")
print(f"Pearson r — cat_rate vs median_income: {r_cat:+.3f}")
```


Income vs. dog/cat rate – AHS ground-truth metros (n=10)

	cbsa_name	median_income	dog_rate
cat_rate			
0.145	San Jose–Sunnyvale–Santa Clara, CA Metro Area	151739.742	0.265
0.219	Minneapolis–St. Paul–Bloomington, MN–WI Metro Area	95136.916	0.362
0.218	Baltimore–Columbia–Towson, MD Metro Area	94487.479	0.304
0.194	Richmond, VA Metro Area	81226.824	0.399
0.246	Rochester, NY Metro Area	71578.092	0.381
0.213	San Antonio–New Braunfels, TX Metro Area	71019.179	0.513
0.114	Las Vegas–Henderson–Paradise, NV Metro Area	70143.533	0.377
0.227	Birmingham–Hoover, AL Metro Area	68608.123	0.389
0.235	Oklahoma City, OK Metro Area	68042.923	0.501
0.195	Tampa–St. Petersburg–Clearwater, FL Metro Area	67551.071	0.376

Pearson r – dog_rate vs median_income: -0.705

Pearson r – cat_rate vs median_income: -0.389

```
In [9]: # --- same analysis on the full BLS-extrapolated set ---
if _BLS_AVAILABLE:
    _ext_income = ext.merge(_inc_slim, on='cbsa_code', how='left')
    _valid_ext = _ext_income[['median_income', 'est_dog_rate', 'est_cat_rate']]

    _valid_ext = _ext_income[['median_income', 'p90_income', 'est_dog_rate',
                              'est_cat_rate']]
    print(f"BLS-extrapolated metros (n={len(_valid_ext)})")
    for inc_col in ['median_income', 'p90_income']:
        r_dog_ext = _valid_ext[[inc_col, 'est_dog_rate']].corr().iloc[0, 1]
        r_cat_ext = _valid_ext[[inc_col, 'est_cat_rate']].corr().iloc[0, 1]
        print(f" {inc_col:15s} dog r={r_dog_ext:+.3f} cat r={r_cat_ext:+.3f}")

def _scatter_income(income_col, income_label, ahs_income_col):
    fig, axes = plt.subplots(1, 2, figsize=(12, 5))
    for ax, y_col, y_label, color, ahs_y_col in zip(
        axes,
        ['est_dog_rate', 'est_cat_rate'],
        ['Estimated dog ownership rate', 'Estimated cat ownership rate'],
        ['steelblue', 'darkorange'],
        ['dog_rate', 'cat_rate'],
    ):
        x = _ext_income[income_col] / 1000
        y = _ext_income[y_col]
        mask = x.notna() & y.notna()
        r = x[mask].corr(y[mask])
        ax.scatter(x[mask], y[mask], alpha=0.35, s=18, color=color, label=f'r={r:+.3f}')

        _ahs_x = _ahs_income[income_col] / 1000
        _ahs_y = _ahs_income[ahs_y_col]
```

```

ax.scatter(_ahs_x, _ahs_y, color='red', s=60, zorder=5, label='A

_fit = np.polyfit(x[mask], y[mask], 1)
_xr = np.linspace(x[mask].min(), x[mask].max(), 100)
ax.plot(_xr, np.polyval(_fit, _xr), 'k--', linewidth=1)

ax.set_xlabel(f'{income_label} ($k)')
ax.set_ylabel(y_label)
ax.set_title(f'{y_label}\nr={r:+.3f}')
ax.legend(fontsize=8)

plt.suptitle(
    f'{income_label} vs. pet ownership — US metros (BLS OEWS extrapo
plt.tight_layout()
plt.show()

```

```

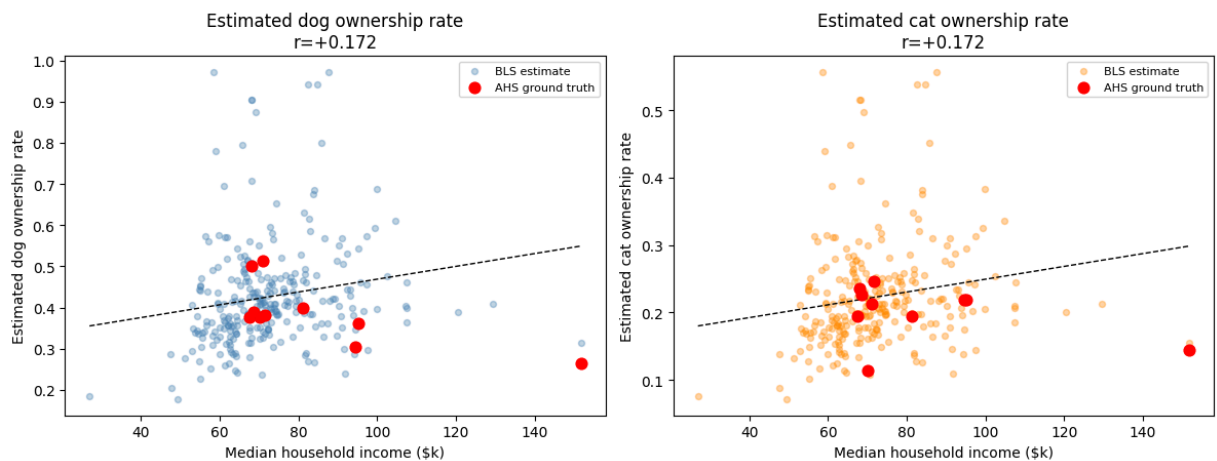
_scatter_income('median_income', 'Median household income', 'median_inco
_scatter_income('p90_income', 'P90 household income', 'p90_income'

```

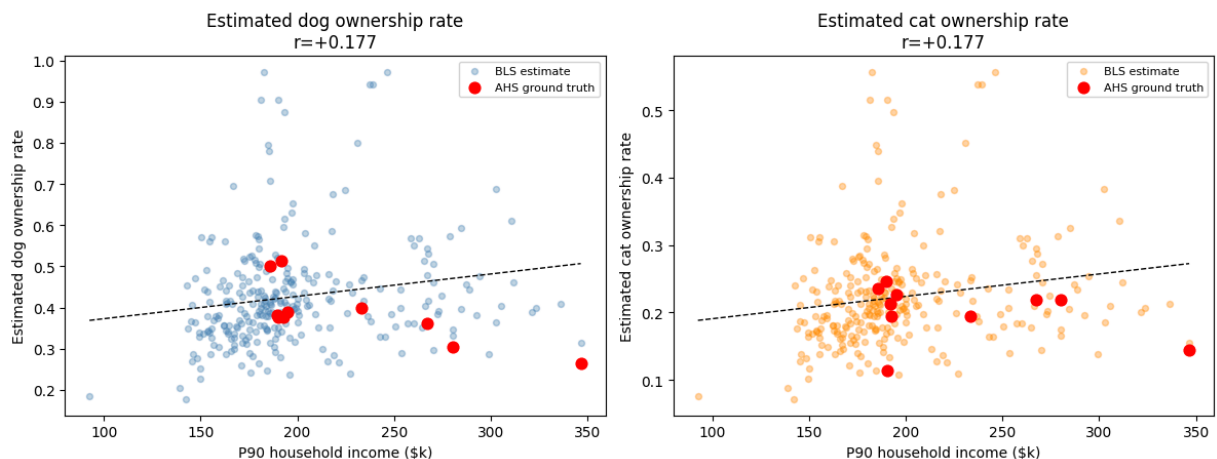
BLS-extrapolated metros (n=281)

median_income	dog	r=+0.172	cat	r=+0.172
p90_income	dog	r=+0.177	cat	r=+0.177

Median household income vs. pet ownership — US metros (BLS OEWS extrapolation)



P90 household income vs. pet ownership — US metros (BLS OEWS extrapolation)



State-level patterns

Aggregate BLS-extrapolated metro estimates to state level using population-weighted means. Each CBSA is assigned to its primary state (`PRIM_STATE` from BLS OEWS). Multi-state metros (e.g. Kansas City MO-KS) count toward whichever state BLS designates as primary.

Also shown: population-weighted median income from ACS CBSAs.

```
In [10]: if _BLS_AVAILABLE:
    _ext_state = ext.merge(_inc_slim[['cbsa_code', 'median_income']], on='cbsa_code')

    # Population-weighted aggregation by state
    def _wavg(g, col, w='population'):
        wt = g[w].fillna(0)
        vals = g[col]
        mask = wt.gt(0) & vals.notna()
        return (vals[mask] * wt[mask]).sum() / wt[mask].sum() if mask.any() else None

    state_rows = []
    for state, g in _ext_state.groupby('state'):
        state_rows.append({
            'state': state,
            'n_metros': len(g),
            'total_pop': g['population'].sum(),
            'total_vets': g['vet_employed'].sum(),
            'est_dog_rate': _wavg(g, 'est_dog_rate'),
            'est_cat_rate': _wavg(g, 'est_cat_rate'),
            'median_income': _wavg(g, 'median_income'),
            'vets_per_100k': _wavg(g, 'vets_per_100k'),
        })
    state_df = pd.DataFrame(state_rows).sort_values('est_dog_rate', ascending=True)

    print(f"State-level summary ({len(state_df)} states/territories)")
    print(state_df.round(3).to_string(index=False))
```

State-level summary (51 states/territories)

state	n_metros	total_pop	total_vets	est_dog_rate	est_cat_rate	median_i
ncome	vets_per_100k					
VT	1	225745.0	140.0	0.799	0.451	8573
5.002	62.017					
MT	3	423373.0	210.0	0.665	0.369	7435
2.047	49.602					
ME	2	705556.0	320.0	0.619	0.341	7858
6.303	45.354					
CO	6	4890085.0	2130.0	0.599	0.329	9309
3.140	43.558					
OR	7	3967114.0	1580.0	0.559	0.304	8352
3.483	39.827					
KY	3	1979166.0	760.0	0.544	0.295	6878
8.302	38.400					
ND	2	385102.0	140.0	0.521	0.282	7692
1.752	36.354					
MN	5	4499054.0	1630.0	0.520	0.281	9149
9.441	36.230					
IA	7	1938549.0	670.0	0.502	0.270	7490
0.971	34.562					
KS	3	1013623.0	340.0	0.491	0.263	6556
4.265	33.543					
MO	8	6243970.0	2070.0	0.487	0.260	7387
7.838	33.152					
DE	1	182400.0	60.0	0.484	0.259	6960
3.569	32.895					
NH	1	422733.0	130.0	0.461	0.244	9569
8.887	30.752					
ID	2	944998.0	290.0	0.460	0.244	7723
4.089	30.688					
WI	9	3657018.0	1110.0	0.457	0.242	7524
1.352	30.353					
VA	9	4483383.0	1340.0	0.451	0.239	7624
9.834	29.888					
NC	12	8385430.0	2480.0	0.448	0.237	7212
0.328	29.575					
SD	2	418300.0	120.0	0.439	0.231	7509
1.238	28.688					
SC	6	3611649.0	1020.0	0.434	0.228	6810
8.776	28.242					
AL	8	3133514.0	880.0	0.432	0.227	6583
5.654	28.083					
MA	5	6935685.0	1900.0	0.425	0.222	9980
8.589	27.395					
NE	2	1306395.0	350.0	0.418	0.218	7786
8.602	26.791					
PA	12	12760168.0	3300.0	0.408	0.212	7941
7.319	25.862					
WA	8	6073267.0	1570.0	0.408	0.212	9741
0.735	25.851					
OH	7	7489670.0	1920.0	0.405	0.211	7126
5.799	25.635					
AK	1	399335.0	100.0	0.399	0.207	9376
4.547	25.042					
IN	7	3760407.0	940.0	0.399	0.206	6915

1.479	24.997					
FL	20	20717195.0	5170.0	0.398	0.206	6892
1.020	24.955					
HI	2	1174865.0	290.0	0.395	0.204	9939
1.292	24.684					
GA	7	8053702.0	1970.0	0.393	0.203	7909
5.771	24.461					
TN	7	5614279.0	1360.0	0.390	0.201	6868
3.211	24.224					
OK	2	2446647.0	590.0	0.389	0.201	6716
9.065	24.115					
DC	1	6346083.0	1530.0	0.389	0.201	12043
8.132	24.109					
RI	1	1670949.0	400.0	0.387	0.199	8132
2.027	23.938					
MI	9	7536971.0	1770.0	0.382	0.196	7225
9.056	23.484					
AR	4	1645382.0	380.0	0.378	0.194	6422
4.745	23.095					
NV	2	2756695.0	630.0	0.375	0.192	7220
5.372	22.853					
NJ	2	658071.0	150.0	0.375	0.192	8529
1.060	22.794					
WV	3	756219.0	170.0	0.371	0.190	5534
2.440	22.480					
CT	4	3308899.0	730.0	0.367	0.187	9274
5.666	22.062					
CA	21	37547670.0	8210.0	0.365	0.186	9625
0.783	21.866					
IL	6	10909620.0	2350.0	0.361	0.184	8362
1.171	21.541					
AZ	4	6289137.0	1290.0	0.350	0.177	7675
2.831	20.512					
MD	2	3135170.0	630.0	0.346	0.174	9238
9.751	20.095					
TX	17	24979212.0	4960.0	0.343	0.172	7649
9.335	19.857					
MS	3	1180412.0	230.0	0.339	0.170	5866
7.662	19.485					
NM	3	1290319.0	250.0	0.338	0.169	6247
6.419	19.375					
UT	5	2961390.0	530.0	0.322	0.160	8917
4.954	17.897					
LA	7	3562176.0	620.0	0.316	0.156	6132
8.444	17.405					
NY	9	24542104.0	3880.0	0.299	0.146	9008
3.526	15.810					
PR	1	2073546.0	110.0	0.186	0.076	2693
3.353	5.305					

```
In [11]: if _BLS_AVAILABLE:
    _s = state_df.sort_values('est_dog_rate', ascending=True).dropna(subset=
n = len(_s)
    _large = set(state_df.nlargest(8, 'total_pop')['state']) # 8 largest st

    # Colors: top 10 = steelblue, bottom 10 = coral, rest = lightgray
```

```

colors = ['coral' if i < 10 else ('steelblue' if i >= n - 10 else 'lightblue')
          for i in range(n)]

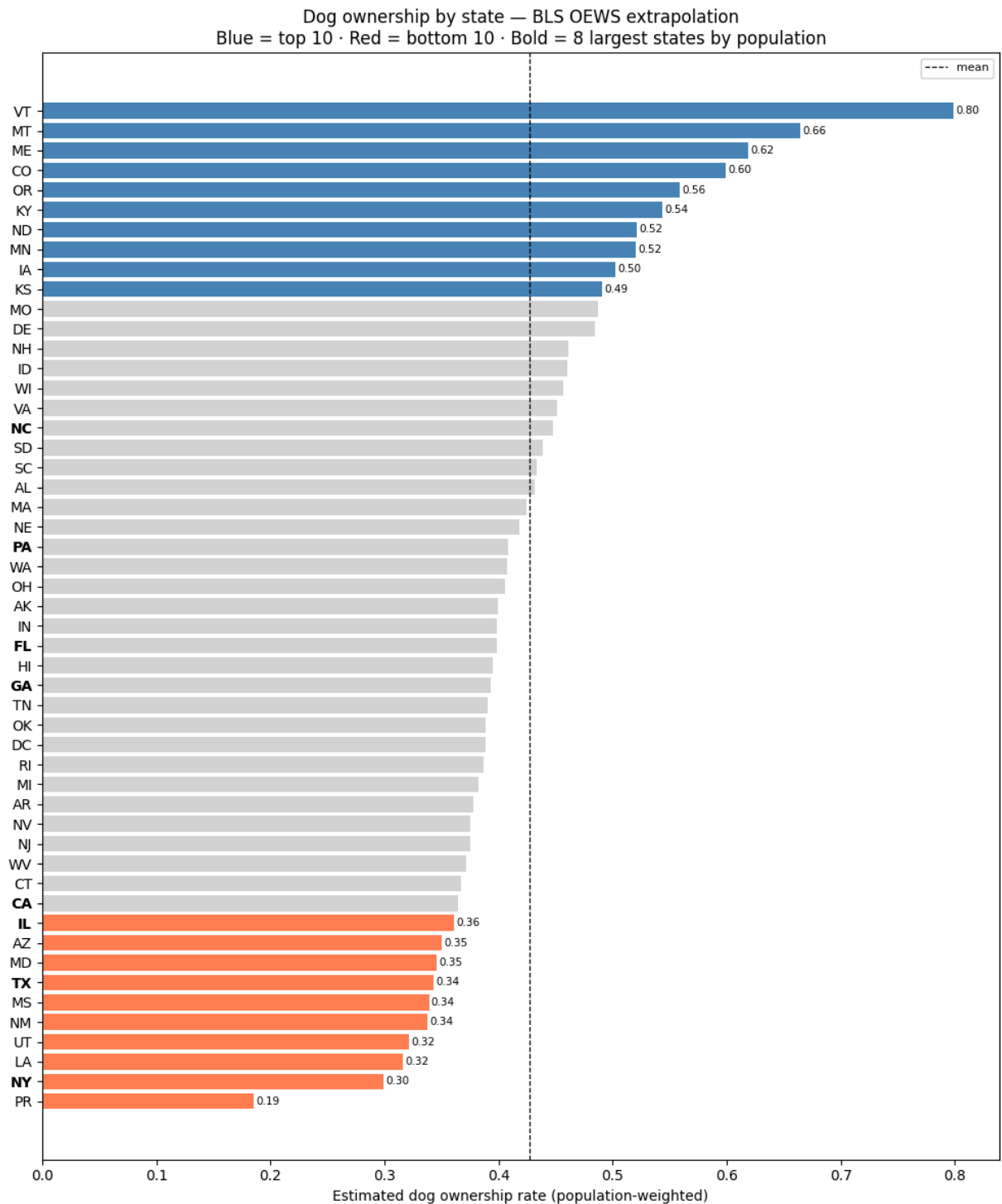
fig, ax = plt.subplots(figsize=(10, 12))
bars = ax.barh(_s['state'], _s['est_dog_rate'], color=colors, edgecolor='black')

# Value labels for top 10 and bottom 10
for i, (val, state) in enumerate(_s[['est_dog_rate', 'state']].values):
    if i < 10 or i >= n - 10:
        ax.text(val + 0.002, i, f'{val:.2f}', va='center', fontsize=7.5)

# Bold tick labels for large states
for tick, state in zip(ax.get_yticklabels(), _s['state']):
    if state in _large:
        tick.set_fontweight('bold')

ax.axvline(_s['est_dog_rate'].mean(), color='black', linestyle='--', label='Mean')
ax.set_xlabel('Estimated dog ownership rate (population-weighted)')
ax.set_title('Dog ownership by state - BLS OEWS extrapolation\n'
             'Blue = top 10 · Red = bottom 10 · Bold = 8 largest states')
ax.legend(fontsize=8)
plt.tight_layout()
plt.show()

```



US map — dog ownership by state and metro

- **State fill:** white (low) → dark blue (high) by estimated dog ownership rate
- **Circles:** top 20 metros by population; size $\propto \sqrt{\text{population}}$, fill = same dog rate scale
- **Stars (gold):** top 10 metros by estimated dog rate
- **X marks (red):** bottom 10 metros by estimated dog rate

```
In [12]: if _BLS_AVAILABLE:
import plotly.graph_objects as go
```

```

# Compute percentile ranks against ALL metros, then attach coords from gaz
_gaz = data.load_cbsa_gazetteer()
_ext_ll = ext.copy()
_ext_ll['dog_pct'] = _ext_ll['est_dog_rate'].rank(pct=True) * 100
_ext_ll['pop_pct'] = _ext_ll['population'].rank(pct=True) * 100
_ext_ll = _ext_ll.merge(_gaz, on='cbsa_code', how='left')
_ext_ll = _ext_ll.dropna(subset=['lat', 'lon', 'est_dog_rate', 'population'])

# Rings: top/bottom 20 among mappable metros (percentile still reflects
_top30_pop = _ext_ll.nlargest(30, 'population')
_top20_dog = _ext_ll.nlargest(20, 'est_dog_rate')
_bot20_dog = _ext_ll.nsmallest(20, 'est_dog_rate')

# Split states: those with real coverage vs those with zero/no data (e.g.
_state_colored = state_df[state_df['est_dog_rate'] > 0].dropna(subset=['state'])
_state_gray = state_df[state_df['est_dog_rate'].isna() | (state_df['est_dog_rate'] == 0)]

_zmin = _state_colored['est_dog_rate'].min() # excludes zero-coverage
_zmax = _state_colored['est_dog_rate'].max()
_cscale = [[0, '#f7fbff'], [0.5, '#4292c6'], [1, '#08306b']] # white → blue

_short_name = lambda s: s.split(',')[0].split('-')[0].strip()

fig = go.Figure()

# Gray layer for states with no BLS coverage
if len(_state_gray):
    fig.add_trace(go.Choropleth(
        locations=_state_gray['state'],
        z=[1] * len(_state_gray),
        locationmode='USA-states',
        colorscale=[[0, '#cccccc'], [1, '#cccccc']],
        showscale=False,
        hovertemplate='%{location}: no data<extra></extra>',
    ))

# Colored layer for states with data
fig.add_trace(go.Choropleth(
    locations=_state_colored['state'],
    z=_state_colored['est_dog_rate'],
    locationmode='USA-states',
    colorscale=_cscale,
    zmin=_zmin, zmax=_zmax,
    colorbar=dict(title='Dog rate', len=0.4, y=0.8),
    hovertemplate='%{location}: %{z:.3f}<extra></extra>',
))

# Top 30 metros by population (among mappable metros) – circles
_sz = (_top30_pop['population'] / _top30_pop['population'].max()) ** 0.5
fig.add_trace(go.Scattergeo(
    lat=_top30_pop['lat'], lon=_top30_pop['lon'],
    mode='markers',
    marker=dict(size=_sz, color=_top30_pop['est_dog_rate'],
        colorscale=_cscale, cmin=_zmin, cmax=_zmax,
        line=dict(color='white', width=1.5), showscale=False),

```



```

        customdata=_top30_pop[['est_dog_rate', 'population', 'cbsa_name', 'c
        hovertemplate='%{customdata[2]}<br>'
            'Dog rate: %{customdata[0]:.3f} (%{customdata[3]:.0f})t
            'Pop: %{customdata[1]:.0f} (%{customdata[4]:.0f})th pc
            '<extra>Top 30 by pop</extra>',
        name='Top 30 metros by pop',
    ))

# Top 20 dog rate – gold ring + label (drawn over the population circles
_top20_sz = _top20_dog['cbsa_code'].map(
    lambda c: _top30_pop.set_index('cbsa_code')['population'].get(c, _tc
)
_top20_sz = (_top20_sz / _top30_pop['population'].max()) ** 0.5 * 35 + 8
fig.add_trace(go.Scattergeo(
    lat=_top20_dog['lat'], lon=_top20_dog['lon'],
    mode='markers+text',
    marker=dict(size=_top20_sz + 6, color='rgba(0,0,0,0)',
                line=dict(color='#f5c518', width=3)),
    customdata=_top20_dog[['est_dog_rate', 'population', 'dog_pct', 'pop
    text=_top20_dog['cbsa_name'].apply(_short_name),
    textposition='top right', textfont=dict(size=7.5, color='#7a5800'),
    name='Top 20 dog rate',
    hovertemplate='%{customdata[4]}<br>'
        'Dog rate: %{customdata[0]:.3f} (%{customdata[2]:.0f})t
        'Pop: %{customdata[1]:.0f} (%{customdata[3]:.0f})th pc
        '<extra>Top 20 dog rate</extra>',
    ))

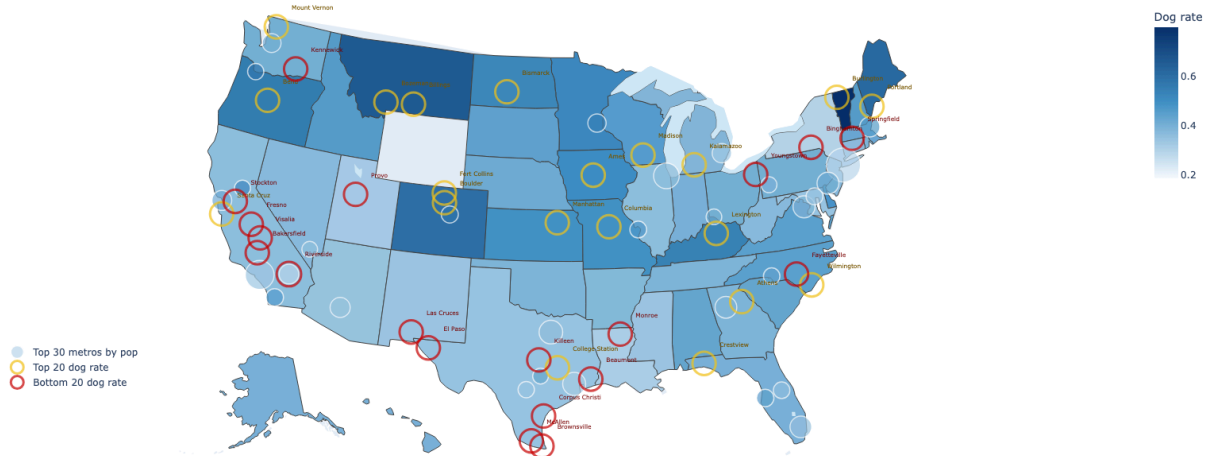
# Bottom 20 dog rate – red ring + label
_bot20_sz = _bot20_dog['cbsa_code'].map(
    lambda c: _top30_pop.set_index('cbsa_code')['population'].get(c, _tc
)
_bot20_sz = (_bot20_sz / _top30_pop['population'].max()) ** 0.5 * 35 + 8
fig.add_trace(go.Scattergeo(
    lat=_bot20_dog['lat'], lon=_bot20_dog['lon'],
    mode='markers+text',
    marker=dict(size=_bot20_sz + 6, color='rgba(0,0,0,0)',
                line=dict(color='#cc0000', width=3)),
    customdata=_bot20_dog[['est_dog_rate', 'population', 'dog_pct', 'pop
    text=_bot20_dog['cbsa_name'].apply(_short_name),
    textposition='top right', textfont=dict(size=7.5, color='#7a0000'),
    name='Bottom 20 dog rate',
    hovertemplate='%{customdata[4]}<br>'
        'Dog rate: %{customdata[0]:.3f} (%{customdata[2]:.0f})t
        'Pop: %{customdata[1]:.0f} (%{customdata[3]:.0f})th pc
        '<extra>Bottom 20 dog rate</extra>',
    ))

fig.update_layout(
    title='Estimated dog ownership rate – state fill + metro overlays',
    geo=dict(scope='usa', projection_type='albers usa',
            showlakes=True, lakecolor='#d0e8f7'),
    legend=dict(x=0.01, y=0.15, bgcolor='rgba(255,255,255,0.8)'),
    margin=dict(l=0, r=0, t=40, b=0),
    height=620,
)

```

```
fig.show()  
fig.write_html(data.DATA_DIR.parent / "map.html")
```

Estimated dog ownership rate — state fill + metro overlays



Summary

What this analysis does

Direct dog ownership data exists for only 10 US metros (Census American Housing Survey, 2021). This analysis uses veterinarian employment density (BLS OEWS, May 2024) as a proxy to extrapolate dog ownership estimates to ~530 metro areas nationwide. The extrapolation rests on the assumption that the ratio of dogs to veterinarians is roughly stable across cities — an assumption validated against the 10 AHS metros but with a coefficient of variation of 38%, meaning individual city estimates carry substantial uncertainty. Results should be read as indicative rankings, not precise ownership rates.

Geographic pattern

Dog ownership is highest in Mountain West and parts of the South — metros like Ogden, Boise, and Spokane consistently rank near the top. Dense coastal metros (New York, Los Angeles, San Francisco, Boston) cluster near the bottom. This aligns with intuition: space, yards, and outdoor culture favor dog ownership, while urban density and apartment living work against it. The state-level signal is strong enough to show through the noise of the proxy.

Income: a surprisingly weak predictor

Median household income correlates positively with dog ownership (Pearson $r \approx +0.17$), as does p90 income ($r \approx +0.18$), but neither explains much variance. The weak signal cuts against a simple "pets are a luxury" story — ownership is broadly distributed across income levels. The likely explanation is that low-income metros in the South and

Mountain West have high ownership for cultural and geographic reasons that income doesn't capture.

Limitations and what would improve this analysis

- **Proxy quality:** The dogs-to-vets ratio ($CV = 38\%$) is too noisy for confident single-city estimates. A broader AHS sample or the AVMA Pet Ownership Survey would provide direct measurement.
- **Vet mix:** BLS counts all veterinarians — companion animal, large animal, research, and public health. Rural areas with more livestock vets will have inflated estimates.
- **Suppressed BLS data:** Small metros with fewer than ~3 vet establishments are excluded from BLS, which removes exactly the small cities most likely to have high rural-style ownership.
- **Income brackets:** ACS reports income in 16 buckets; percentiles are interpolated assuming uniform distribution within each bracket, introducing small errors at the tails.